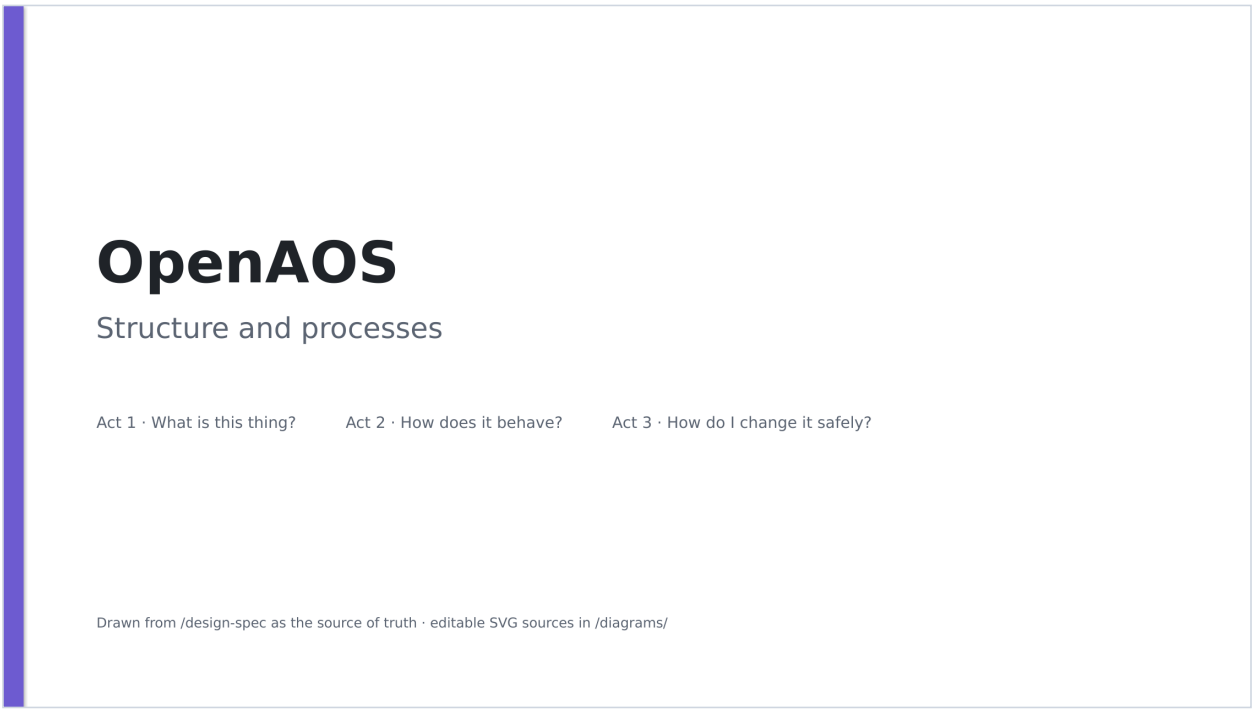
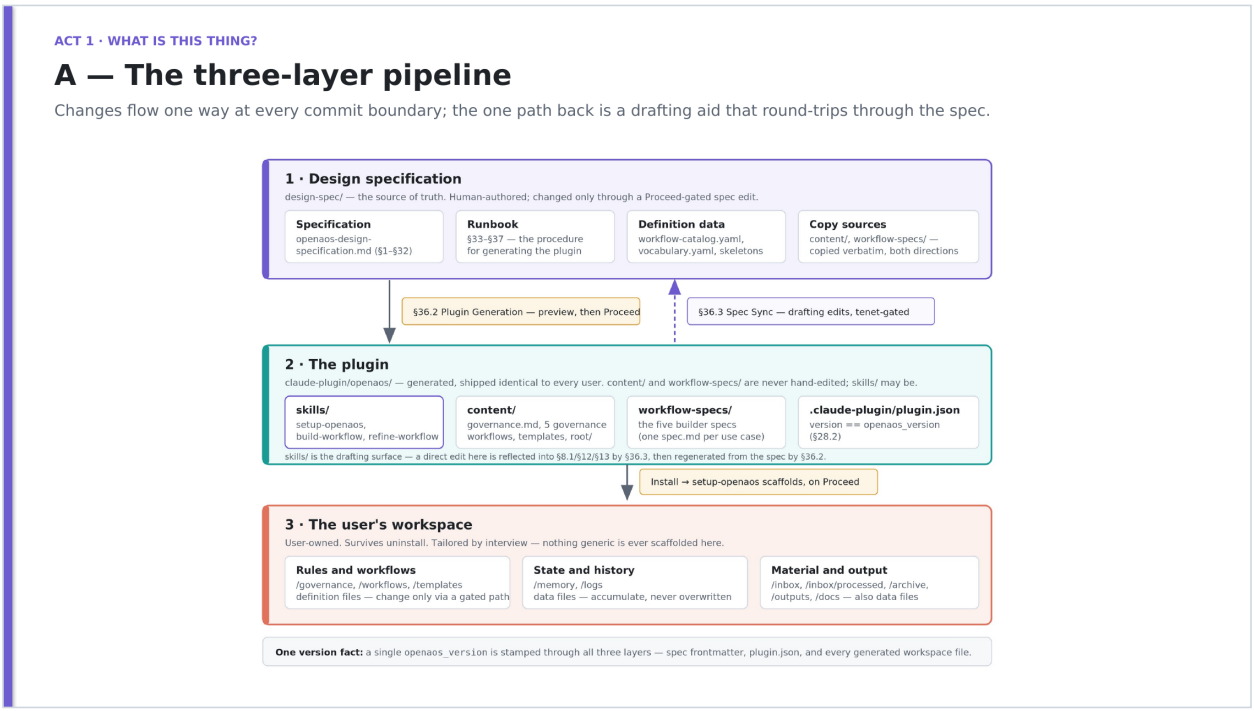
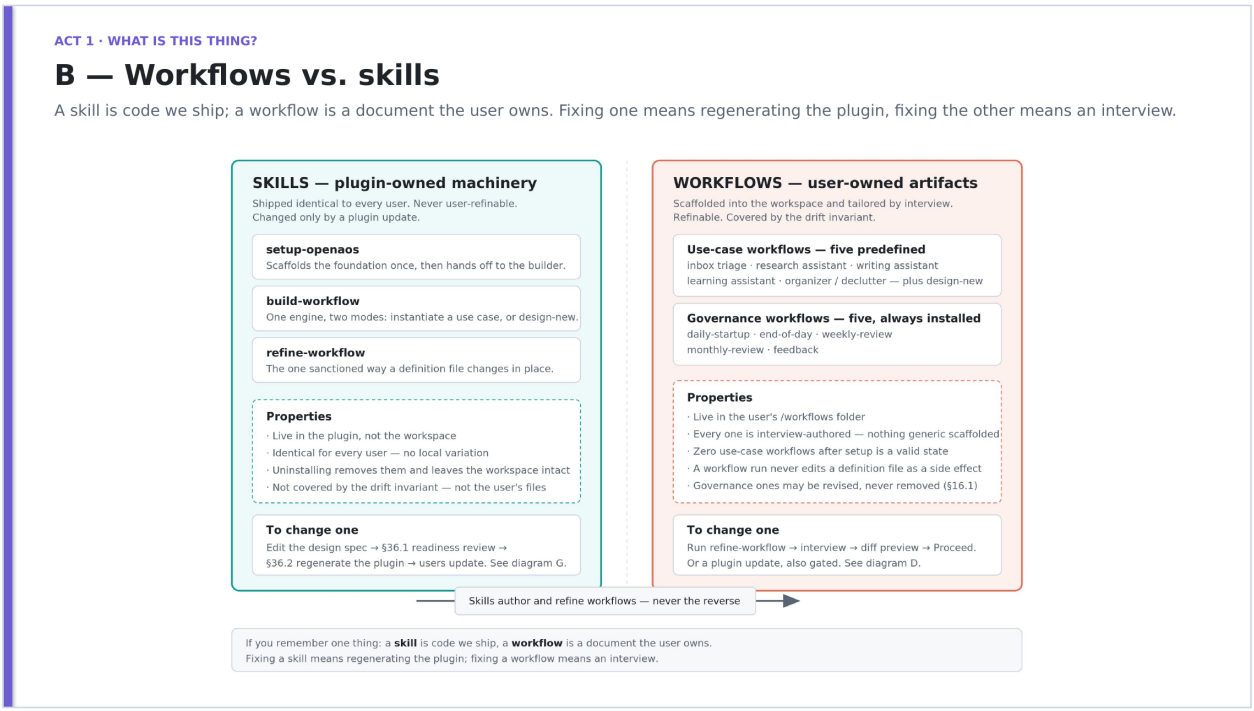


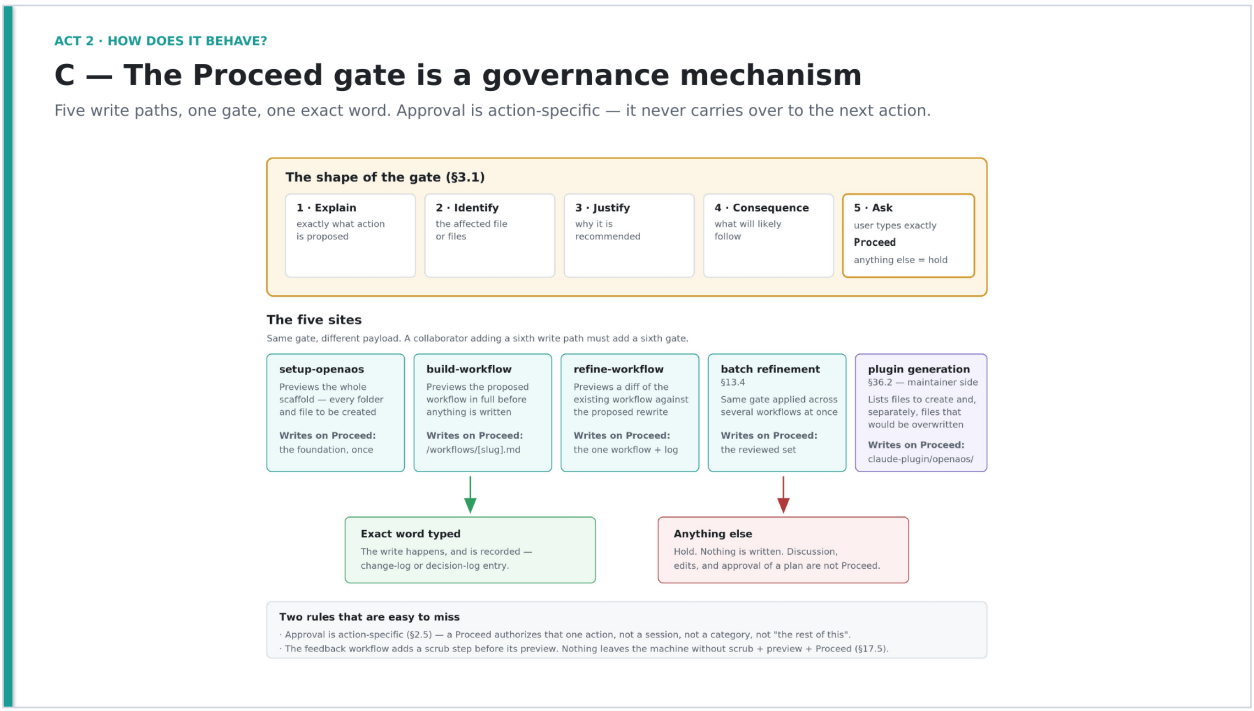
This deck introduces OpenAOS to someone who will be working on it but has not seen it before, and it is arranged as three questions rather than a tour of the file tree. Act 1 establishes what the project is: a design specification that generates a plugin, which in turn scaffolds a workspace the user owns. Act 2 covers how the resulting system behaves day to day — where it stops and asks permission, how a single workflow evolves, and what runs on a schedule. Act 3 covers the part that matters most to a contributor: how to change any of it without breaking the guarantees the earlier slides describe. Every diagram was drawn from design-spec/ as the source of truth, and the editable SVG sources sit alongside the deck in /diagrams/ — the images here are rendered PNGs, so amend the SVG and regenerate rather than editing inside PowerPoint.



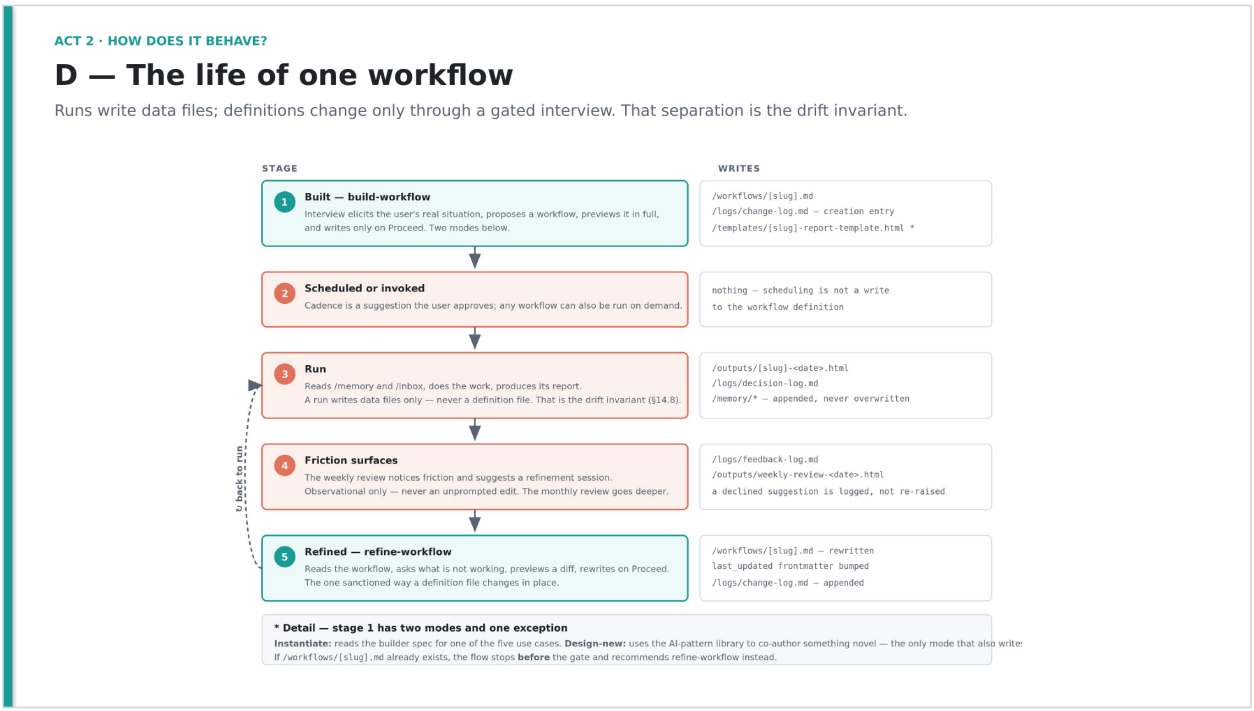
Three layers, and changes flow downward at every commit boundary. The design specification is human-authored and is the single source of truth; the plugin is generated from it; the user's workspace is scaffolded by the plugin at install time and thereafter belongs entirely to the user. Three things are worth pausing on. First, the content/ and workflow-specs/ folders are copied into the plugin verbatim rather than re-rendered, which is why generation can verify them with a byte-for-byte diff — and why the same copy can run in reverse during a spec sync. Second, skills/ is the one exception to “never hand-edited”: the plugin copy may be edited directly as a drafting surface, because editing a SKILL.md is concrete in a way that editing §12 of a large specification is not. That edit is not a shortcut into main — runbook §36.3 reflects it back into §8.1/§12/§13, the author reviews and owns the resulting spec prose, and §36.2 then regenerates the skill file from the spec, round-tripping the prototype away. Third, a single openaos_version is stamped through all three layers — spec frontmatter, plugin.json, and every generated workspace file — so there is exactly one version fact to reason about rather than a compatibility matrix. The practical consequence for a contributor is unchanged: what you propose is always a change at the top of this diagram.



This is the distinction that governs every other conversation about the project, and confusing the two is the most common way to misunderstand a design decision. The plug-in contains three skills: /setup-openaos, /build-workflow, and /refine-workflow. These skills are plugin-owned machinery. They ship identical to every user, they are never user-refinable, and they change only through a plugin update. Workflows are user-owned documents that live in the user's /workflows folder, are authored through an interview so that no two users get the same file. Workflows are refinable at any time. Note the asymmetry at the bottom: skills author and refine workflows, never the reverse. The operational shorthand is that a skill is code we ship and a workflow is a document the user owns, which means fixing a skill requires regenerating the plugin while fixing a workflow requires nothing more than an interview using the /refine-workflow skill.



The system writes something irreversible in exactly five places, and all five stop at the same gate rather than each inventing its own confirmation style. The gate has a fixed five-part shape drawn from design-spec section §3.1: explain the action, identify the affected files, justify it, state the likely consequence, and ask the user to type the exact word. Anything other than that exact word is a hold — discussion, edits, and even enthusiastic approval of a plan are not Proceed. Two rules are easy to miss and worth stating aloud. Approval is action-specific under §2.5, so a Proceed authorizes one action and does not carry forward to a session or a category. And the feedback workflow inserts a scrub step before its preview, because nothing leaves the user's machine without scrub, preview, and Proceed — that sequence is an important privacy boundary.

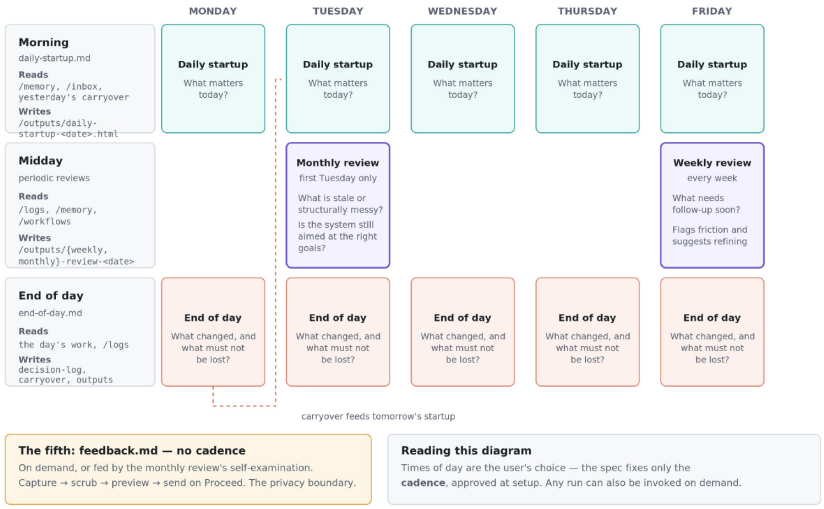


A workflow is not written once and left alone. It is expected to change over time to improve its behavior and adapt to changing requirements. This diagram shows the sanctioned path for that change alongside the artifacts each stage produces. It is built through an interview, either scheduled or invoked, run repeatedly, observed for friction by the weekly and monthly reviews, and eventually refined. This is an endless learning loop. Notice that runs write only data files while the workflow definition itself is untouched. That separation is the drift invariant from design-spec §14.8, and it is what keeps updates and refinements clean reviewed changes rather than three-way merges. Two details at the bottom are easy to trip over: 1) only design-new mode also writes a report template, and 2) if the target workflow file already exists the builder stops before the gate and recommends refinement instead.

ACT 2 · HOW DOES IT BEHAVE?

E — The governance rhythms

Five governance workflows on four cadences — what the system does when nobody asks it to do anything.

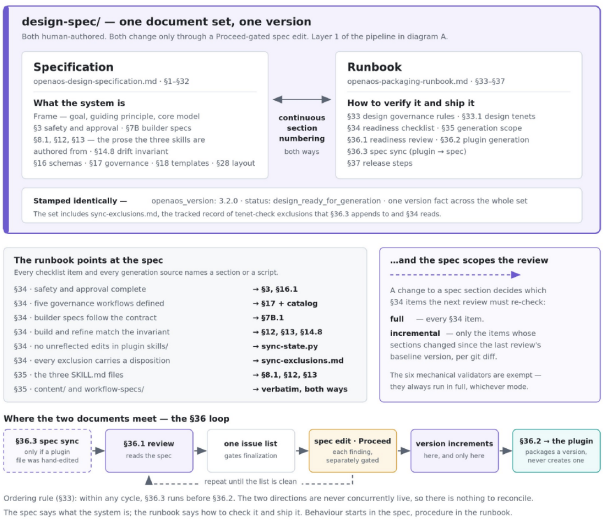


This is what the system does automatically on a pre-defined schedule. Five governance workflows run on four cadences: a daily startup that asks what matters today, an end-of-day pass that captures what changed and what must not be lost, a weekly review that surfaces follow-ups and flags workflows producing friction, and a monthly review that asks the deeper question of whether the system is still aimed at the right goals. The dashed line traces the carryover from one day's close into the next day's startup, which is the mechanism that keeps obligations from evaporating overnight. The specification fixes only the cadence, not the clock times — those are proposed at setup and approved by the user. Scheduled workflows can also be invoked on demand. The fifth workflow, feedback, has no cadence at all and is triggered on demand or by the monthly review's self-examination.

ACT 3 · HOW DO I CHANGE IT SAFELY?

F — Specification and runbook

Two peer documents, one version. The spec says what the system is; the runbook says how to verify it and ship it.

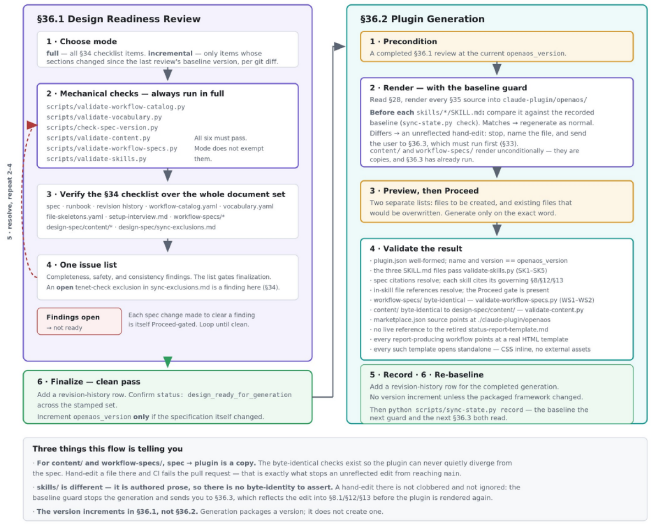


The design spec and the runbook are peers, not layers. They are both human-authored, neither is generated from the other, and their section numbering is deliberately continuous (spec §1–§32, runbook §33–§37) so cross-references resolve in both directions. The division is that the specification says what the system is and the runbook says how to verify it and ship it, which means a change in behavior starts in the spec and a change in procedure starts in the runbook. The middle band is worth reading aloud: every §34 checklist item and every §35 generation source names a specific spec section or the script that checks it, so the runbook never restates the design, it only points at it. The traffic runs the other way too, because which spec sections changed is exactly what decides whether the next review is full or incremental. The bottom strip is the loop where the two documents actually meet. It now opens with an optional first step: if a plugin file was hand-edited, §36.3 runs before anything else, because the ordering rule in §33 says the two sync directions are never concurrently live. The rest of the strip contains the detail people get wrong most often — findings from the review become Proceed-gated edits to the spec, and the version increments during the review, not during generation. Generation packages a version; it never creates one. One maintenance note when presenting: the stamp band shows openaos_version 3.2.0 as an illustration of the single version fact, and it will need updating whenever the set is re-stamped.

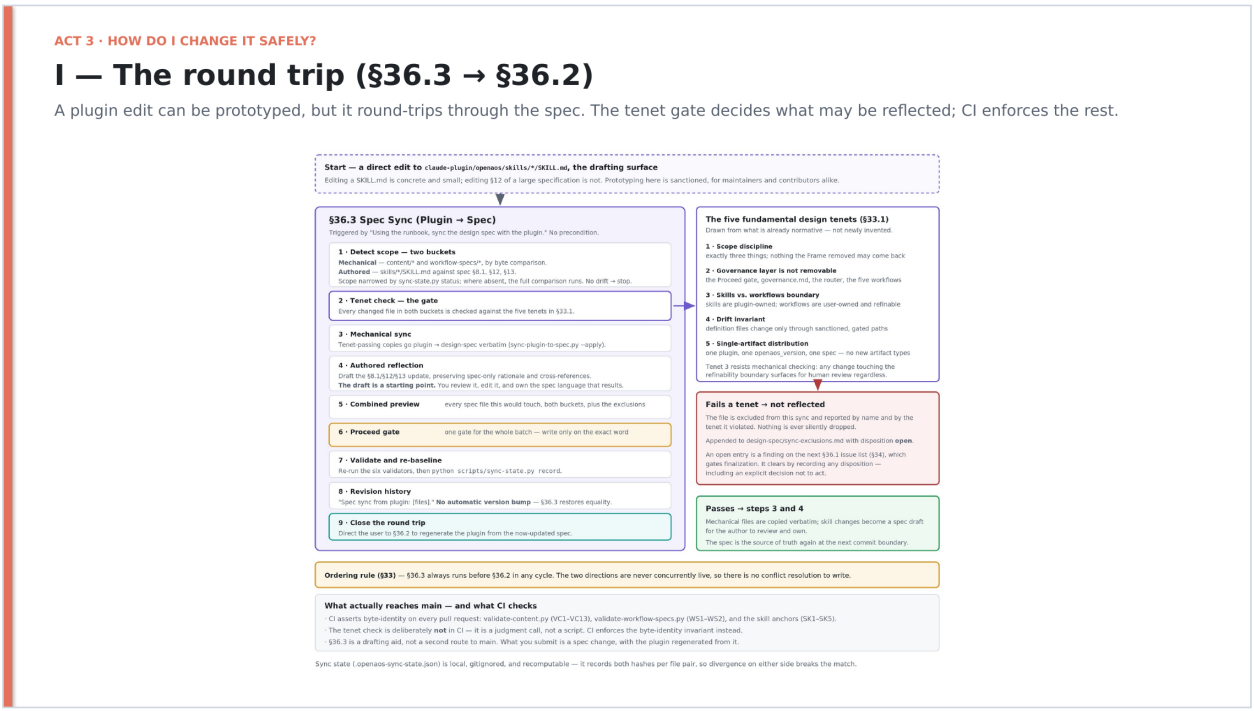
ACT 3 · HOW DO I CHANGE IT SAFELY?

G — Readiness review → plugin generation

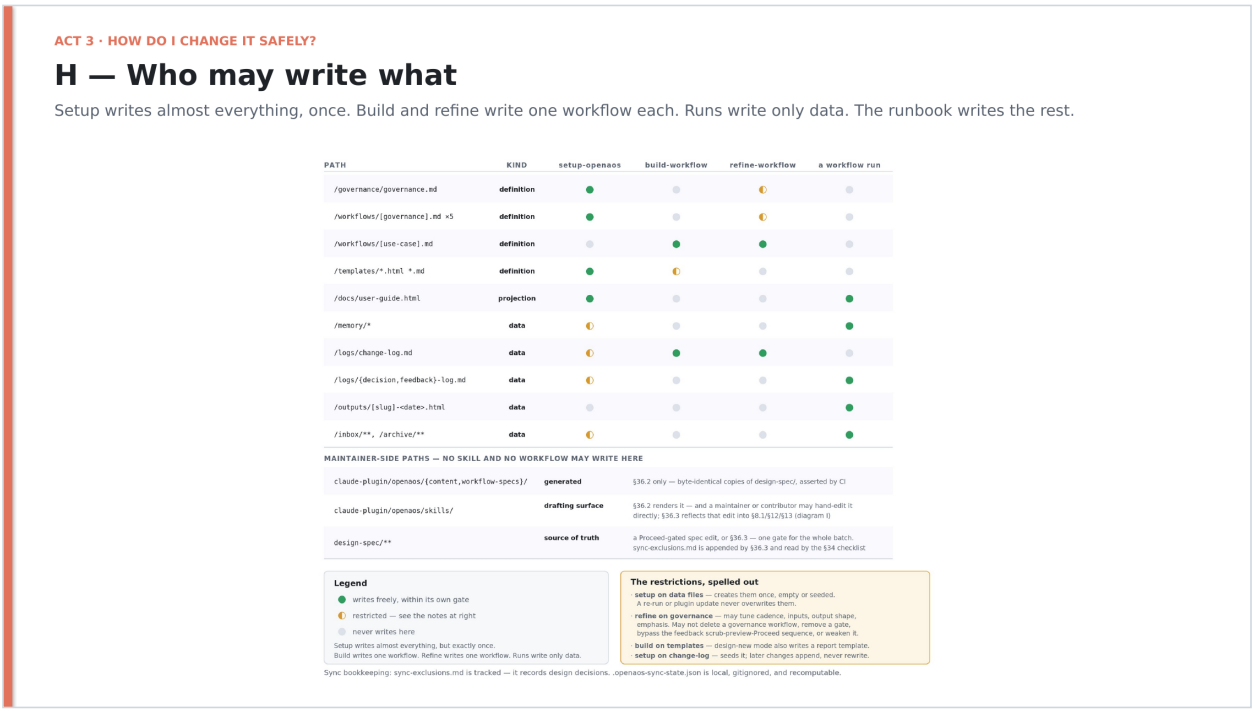
Generation cannot start without a clean review at the current version — and the baseline guard stops it clobbering an unreflected plugin edit.



This is the maintainer's flow, and it is drawn as one diagram rather than two because the second half cannot start without the first: runbook §36.2 step 1 is a precondition requiring a completed §36.1 review at the current version. The review runs six mechanical validators — which execute in full regardless of whether the review itself is full or incremental — then verifies the §34 checklist across the whole document set, collects everything onto one issue list, and loops until that list is clean, with each remedial spec change itself Proceed-gated. An exclusion in sync-exclusions.md whose disposition is still open counts as a finding on that list, so an undecided conflict between a plugin edit and a design tenet does not stop a review from running, only from finalizing. Generation then renders, previews, and validates. Two validation items are structural rather than cosmetic: content/ and workflow-specs/ must be byte-identical to their design-spec sources, and CI asserts the same on every pull request — that is what makes an unreflected plugin edit unable to reach main. skills/ works differently, because it is authored prose with no byte-identity to assert. Before rendering each SKILL.md, generation compares it against the recorded sync baseline; if it has been hand-edited, generation stops and sends you to §36.3 rather than clobbering the edit. Note also that the version increments during the review, not during generation.



This is the path the bidirectional-sync change added, and it exists because of an honest observation about productivity: editing a SKILL.md is concrete and small, while editing §12 of a large specification is not. For a maintainer that difference is a tax; for an outside contributor it was a barrier to contributing at all. §36.3 makes the plugin copy of a skill a sanctioned drafting surface — but it does not create a second way for a change to reach main. The submission contract is unchanged: what you propose is a spec change, and the plugin in your pull request must be byte-identical to what the spec generates, which CI checks. Two decisions carry most of the weight. The first is the ordering rule — §36.3 always runs before §36.2 within a cycle, so the two directions are never live at the same moment. That removes the question of which side is authoritative, and it means there is no conflict-resolution logic anywhere in the system. The second is the tenet gate. Before any plugin-side change is reflected into the spec it is checked against five tenets already normative elsewhere in the design: scope discipline, the non-removable governance layer, the skills/workflows boundary, the drift invariant, and single-artifact distribution. A change that fails is not reflected and not silently dropped — it is named with the tenet it violated, appended to sync-exclusions.md with disposition open, and it becomes a finding on the next readiness review until a decision is recorded, including a decision not to act. The tenet check is deliberately not automated into CI, because it is a judgment call; CI's job is the narrower, mechanical one of byte-identity.



This slide is a reference table rather than a narrative, and it answers the question a new contributor asks when they are about to write code: am I allowed to touch this path? Read the KIND column first, because it determines the rules — definition files change only through a gated path, and data files accumulate and are never overwritten by an update. The shape that emerges in the top half is a clean division of labor between the plugin skills: setup-openaos writes almost everything but exactly once, build-workflow and refine-workflow each write a single workflow file plus a log entry, and workflow runs write only data. The amber panel spells out the four restricted cases, of which the most consequential is refinement's boundary on governance — it may tune cadence, inputs, output shape, and emphasis, but it may not delete a governance workflow, remove an approval gate, bypass the feedback scrub sequence, or weaken the governance config. The bottom block is the maintainer's side of the line, and no skill and no workflow run writes anywhere in it: content/ and workflow-specs/ are written only by §36.2, design-spec/ only by a Proceed-gated spec edit or by §36.3, and skills/ is the single path where a direct hand-edit is sanctioned — as a drafting surface that §36.3 round-trips back into the spec.